REVIO STOCKSENSE

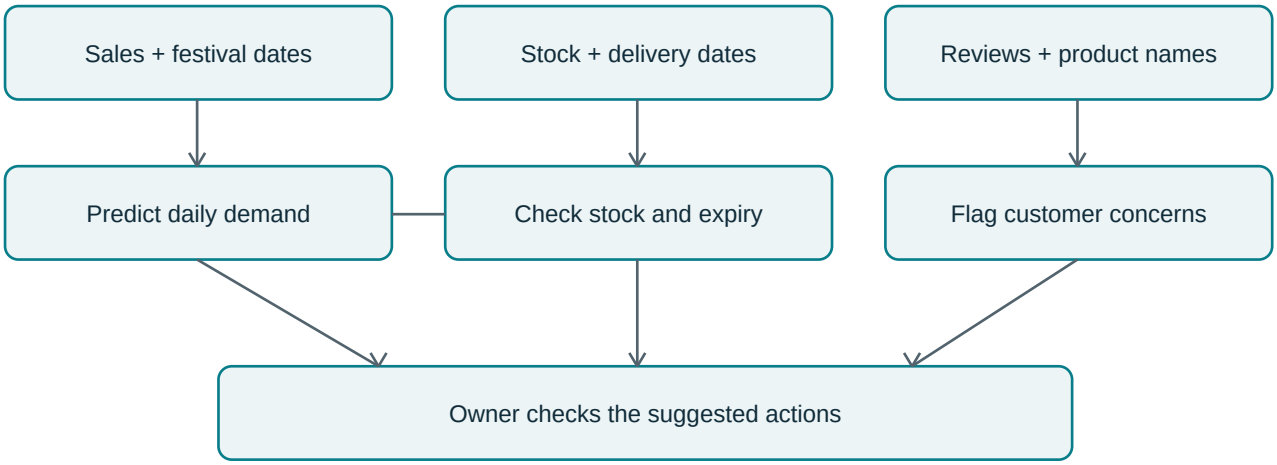
AI Opportunity Report

The shop problem, who could use the app, and what still needs testing.

17 October 2026
Independent proposal for Cynbit AI/ML Engineer Internship

The shop data is generated by a program. Real-shop accuracy and savings are not yet known. The app does not connect to Revio or place orders.

System at a glance



A simple explanation of the business idea. Read this first to understand why the project exists.

What problem does it try to solve?

A shop can run out of a product that customers want while keeping too much of another product. Some of that extra stock may expire. StockSense tries to help an owner decide what to buy, what to stop buying, and which stock needs attention.

It estimates demand for the next 7, 14 or 30 days. It then checks current stock, incoming deliveries and expiry dates before suggesting an order quantity. The owner makes the final decision. The app does not place orders.

How does this relate to Cynbit and Revio?

The company's public pages, checked on 17 October 2026, describe software, AI, cloud and marketing services. They list Revio for customer-review work, TenantEase for property and rent work, and Studio for coding and career support [1, 2]. These are the company's descriptions; we did not independently test every feature.

Revio advertises review collection, AI replies, customer sentiment, issue alerts and multiple-location roles. StockSense is a separate assessment proposal that could be discussed as an addition. It has no working connection to Revio. We have not established whether Cynbit already has a similar inventory feature privately.

The proposed users are kirana owners and people who buy stock for shops. They are not verified existing Revio customers. Before building a real business around this idea, ask Cynbit about feature overlap, access to sales data and actual customer interest.

Other ideas considered

Idea	What it would help someone do
Stock and expiry planning	Decide what to reorder and which batches may remain unsold
Review-reply checking	Find AI replies that are inaccurate or unsuitable before sending them
Repeated complaint tracking	Notice the same customer problem appearing again and again
Rent-payment checking	Flag unusual or missing payments for a person to review
Maintenance-request grouping	Put similar property repair requests together
Learning and interview feedback	Help learners understand mistakes and choose practice tasks

These are possible improvements to explore, not claims that Cynbit lacks them. StockSense was chosen because it makes it possible to demonstrate a prediction and then explain the stock decision built from it.

What is useful about the approach?

A demand forecast alone is not enough. A delivery may arrive too late, stock may expire, or a supplier may sell only in packs. StockSense shows these reasons alongside the suggested quantity. Customer reviews can raise a question, such as a request for sugar-free cola, without being converted into invented sales.

Inventory alerts and batch tracking already exist in other products. Zoho's documentation describes reorder levels and batch dates [3, 4]. ONDC's DigiDukaan page discusses digital tools for shop ordering and stock work [5]. These sources support the problem context. They do not prove this project is new or better.

Which model does the app use?

The current demo uses **Seasonal Poisson Regression**. The Python class is called `PoissonRegressor`, from the scikit-learn library. A separate model is trained for each product that has enough sales history.

A regression model predicts a number. Here, that number is the expected units sold per day. Poisson regression is designed for counts and keeps the expected number above zero. A result such as 7.5 means an average expectation, not that the shop can sell half a packet.

The model learns four kinds of patterns together: changes over the year, differences between weekdays, slow growth or decline over time, and festival preparation days. These produce **one forecast**, which the ordering and expiry pages also use.

The annual pattern uses sine and cosine waves. These are smooth repeating curves that help the model move gradually from one season to the next. The model does not simply apply a rule such as "increase every product in winter."

Festival preparation means the seven days before a festival plus the festival day. For each product, the current model learns one shared festival effect from all past festival windows. It does not learn a separate Diwali effect and Holi effect. We made this choice because there are only two or three past examples of each named festival. Separate effects were reacting too strongly to random variation.

Why choose this model?

It fits the kind of number we want to predict, can learn a different seasonal pattern for each product, and runs on a normal laptop. More importantly, it had the lowest average error in the earlier periods used to choose the model. This is a reason to use it in this demo, not proof that it is the best model for a real shop.

The app also compares it with three other methods. A 28-day average repeats the recent average. A weekly repeat method copies the last week's pattern. HistGradientBoosting is a machine-learning model that combines many small decision trees to learn more complex relationships.

When does the app avoid this model?

The seasonal model needs at least 180 usable daily observations spread across at least 365 days. Otherwise, it uses the 28-day average and labels that choice. If even recent sales history is too limited, it gives no exact forecast. A new product does not get a made-up demand number just because a review asks for it.

How accurate is it?

Real-shop accuracy is not known. The following numbers come from the project's computer-generated data. We should not present them as results from Cynbit customers or real shops.

Method	Error used to choose the model	Error in later test periods
Seasonal Poisson Regression	3.088 units/day	2.860 units/day
HistGradientBoosting	3.178 units/day	2.964 units/day
28-day average	3.507 units/day	3.374 units/day
Repeat the last week	4.230 units/day	4.133 units/day

Both error columns use **MAE: Mean Absolute Error**. It is the average size of a mistake, ignoring whether the forecast was too high or too low. If the actual sale is 10 and the forecast is 7, the error is 3 units. An MAE of 2.860 means the forecast was wrong by about 2.86 units per evaluated product per day on average. It does not mean every forecast is within that amount.

Other result	Value	What it means
WAPE	18.520%	Add all absolute errors, then divide by all actual units sold. Lower is better.
RMSE	3.921 units	An error score that gives extra weight to large mistakes. Lower is better.

Other result	Value	What it means
Bias	-0.040 units	Average forecast minus actual sales. The small negative value means slight overall underprediction.

Do not call this 81.48% accuracy. Subtracting WAPE from 100 does not create a reliable "percent correct" score. Forecasting predicts quantities, so the size of each error matters.

On the same current dataset, the chosen model's MAE is about 3.5% lower than boosting and 15.2% lower than the average method. That is a comparison of forecast errors, not a percentage of money saved. A three-unit error may be small for a product selling 50 units a day but serious for one selling four.

How were these numbers measured?

First, the models learned from older records and predicted four later 30-day periods. Those periods were used to choose the model. Next, the chosen method was kept fixed while all methods were tested over twelve more 30-day periods, covering 360 days.

At each step, training used only records from before the forecast date. As time moved forward, training could include records that had become available since the previous step. This imitates updating a model as a shop collects more sales.

The main test contains **4,200 product-days**. One product-day means one product on one date. There were 4,320 possible records for the 12 products across 360 days; 120 were left out because sales were limited by stock availability. The thirteenth product has no sales history and is not scored.

This project has been developed and adjusted using the same simulation. It is not an independent real-world test. Earlier versions also used different data, so their scores should not be used to claim a clean before-and-after improvement.

What is in the dataset?

The shop records are **synthetic**, which means a Python program created them. They are not downloaded from a real store and are not Cynbit data. The festival dates are the exception: they were imported from the public Google Calendar called Holidays in India.

Item	Current dataset specification
Store coverage	One fictional kirana store
Sales dates	1 January 2024 to 16 October 2026
History length	1,020 days for each of 12 products
Products	13 listed products; 12 have sales and one is a new product
Sales records	12,240 rows: one product on one day per row
Stock batches	24 rows describing stock held at the latest snapshot
Purchase orders	3 examples of incoming or planned supply

Item	Current dataset specification
Customer reviews	14 written examples, not real customer messages
Festival dates	28 entries: seven festival names across 2024-2027
Calendar import date	17 October 2026; saved copy, not automatic live sync
Dataset version	winter-calendar-v3
Random seed	42; this makes the same generator repeatable

Products include cola, water, mango drink, milk, yogurt, chips, tea, coffee, biscuits, chocolate, packaged sweets and rice. Sugar-free cola is the new product without sales history. Prices and stock values are examples in Indian rupees.

The generator combines a starting demand level with seasonal changes, weekends, gradual trends, festival periods, example promotions and random variation. It also creates supply interruptions so some days run out of stock. It deliberately includes expiry and overstock examples so those screens can be demonstrated.

The current tea and coffee example has a winter peak. The earlier demo curve peaked near October by mistake; the generator was corrected. That change is an assumption in a fictional example, not something discovered in real shop records. The gifting example also has a festival increase built into the generated demand. A model matching these patterns does not prove that it will match a real shop.

Why this data is a major limitation

Real demand can change with location, prices, promotions, weather, competitors, customer habits and supply problems. The small simulation does not capture all of that. A good-looking test result can therefore coexist with forecasts that feel unrealistic.

We need real sales and stock records, with permission to use them, before judging business usefulness. Those records should include dates, product quantities, price changes, promotions and stock availability. We should test future periods that were not used while adjusting the model, and inspect errors for each product, not only one average score.

What would success look like in a real shop?

Useful measures would include fewer hours out of stock, less money lost on expired goods, less money tied up in slow stock, and less time spent checking orders. None of these benefits has been measured in this project.

A first trial should show suggestions to the owner without placing orders. Compare the suggestions with the shop's current method over several periods. Record what the owner accepts, what they reject and why. Also record whether supply problems, promotions or price changes explain the outcome.

What should happen before real use?

1. Confirm the idea and any existing feature overlap with Cynbit.
2. Obtain permission to use real shop records and check that stock balances are correct.
3. Compare models on later real periods and examine mistakes by product and season.
4. Try recommendations with owner review and measure stockout and expiry costs.
5. Add sign-in, separate each shop's records, and keep a record of changes before wider use.

Where to read next

The Technical Approach Report explains the model, data, tests and order calculation. The Complete Project Explanation explains each screen and the files in the repository, with short interview answers.

Sources checked on 17 October 2026

[1] Cynbit services: https://www.cynbit.in/services

[2] Cynbit products: https://www.cynbit.in/products

[3] Zoho reorder levels: https://www.zoho.com/inventory/help/items/additional-features.html

[4] Zoho batch tracking: https://www.zoho.com/us/inventory/help/advanced-inventory-tracking/batch-tracking.html

[5] ONDC DigiDukaan: https://www.ondc.org/pages/digidukaan.html